

Review Cheat Sheet — A Readable Companion to the OTP Docs

Chapter 14 · all checklists in one place · Sep 25, 2026

How to use this sheet

This is every chapter’s checklist, regrouped by the question a reviewer asks rather than by topic. The number after each item is the chapter that explains it. Use the first section to find suspicious code quickly, and the rest to review it properly.

1. Red flags you can search for

These patterns are not always wrong, but each one deserves a second look.

Search for	Why it’s suspicious	Ch.
<code>timer:sleep</code> / <code>Process.sleep</code> in tests	Synchronising by waiting; flaky under load	8
<code>list_to_atom</code> , <code>binary_to_atom</code> , <code>String.to_atom</code>	Atoms from input exhaust the atom table	3, 12
<code>os:cmd</code> with <code>++</code> or string interpolation	Shell injection	11
<code>spawn()</code> without <code>link</code> , <code>monitor</code> or a supervisor	Failures go unnoticed	2
<code>++</code> inside a fold or recursion	Quadratic copying	3
<code><<X/binary, Acc/binary>></code>	Prepending forces a full copy each time	3
<code>ets:tab2list</code> , <code>match_object</code> with <code>'_'</code> keys	Full table scans	4
<code>ets:lookup</code> followed by <code>ets:insert</code> on the same key	Lost updates under concurrency	4
<code>persistent_term:put</code> outside startup code	Global garbage collection on every update	4
<code>{noreply, State, Timeout}</code>	Reset by every message	5
<code>receive</code> inside a <code>gen_statem</code> or <code>gen_server</code> callback	Blocks the machine; can swallow system messages	13
<code>{{timeout, Name}, ...}</code> without a matching cancel	Named timers survive state changes	13

Search for	Why it's suspicious	Ch.
<code>gen_server:call</code> inside <code>handle_call</code>	Blocks the server; possible call cycles	5
<code>catch exit:{timeout, _}</code> followed by a retry	Timeouts are unknown outcomes; duplicates	2
'DOWN' or <code>nodedown</code> handling that takes over leadership	Split brain during partitions	6
Anonymous funs passed to <code>rpc / erpc</code>	<code>badfun</code> across code versions	6
<code>application:set_env</code> outside tests	Not a runtime control channel	7
<code>ensure_all_started(App)</code> in production boot code	Temporary by default; node survives a dead app	7
<code>process_info(Pid, messages)</code>	Copies the entire mailbox	9
<code>sys:get_state</code> outside a shell session	Debug-only, times out under load	9
<code>dbg:p(all, ...)</code>	Traces every process	9
<code>logger:debug(io_lib:format(...))</code>	Formats even when filtered out	9
NIF flags <code>0</code> on long work, or <code>DIRTY_JOB_IO_BOUND</code> on computation	Blocks or starves schedulers	10
<code>static ERL_NIF_TERM</code>	Term kept after its environment died	11
<code>Task.async</code> inside a <code>GenServer</code> callback	Linked crash plus a blocked server	12
<code>@x Application.get_env(...)</code>	Config frozen at compile time	7, 12

2. What happens when something crashes?

- ❑ Child order matches dependency order, and the strategy reflects it (`rest_for_one` where later children need earlier ones) (1)
- ❑ Every child supervisor has `type => supervisor` (1)
- ❑ Per-request and per-connection children are `temporary` (1, 12)
- ❑ `intensity/period` are set deliberately, as burst size and sustained rate, and not identical at every level (1)
- ❑ Work-unit supervisors use `auto_shutdown` and are not `permanent` in their parent (1)
- ❑ Every spawned process is linked, monitored or supervised (2)
- ❑ Every ETS table's owner is chosen deliberately (dedicated process or `heir`) (4)
- ❑ `start/2` only starts the top supervisor; production applications are `permanent` (7)
- ❑ Recovery is tested by killing processes, not only by starting the tree (8)

3. What happens when it shuts down?

- ❑ Workers with cleanup trap exits and have an integer `shutdown` (1, 5)
- ❑ Port programs exit when stdin closes; no orphaned OS processes (11)

4. What happens under load?

- ❑ `init/1` returns fast; slow setup happens in `handle_continue/2` (1)
- ❑ No callback does slow or blocking work inline; slow work replies later with `gen_server:reply/2` (5)
- ❑ Every hand-written `receive` has a catch-all, or is replaced by a behaviour (2, 5)
- ❑ `gen_statem` has a final clause for unexpected `info` events; specific clauses come before generic `postpone` (13)
- ❑ No `receive` inside behaviour callbacks; waiting is modelled as a state (13)
- ❑ High-volume producers have backpressure (`call`, batching or shedding) (5)
- ❑ CPU-heavy work is spread across processes, not funnelled through one server (10)
- ❑ High-traffic processes have `message_queue_data` considered; heap tuning is measured (2)
- ❑ No log line formats whole states, payloads or unbounded terms (9)

5. What happens with concurrent access?

- ❑ No code relies on message ordering except between one sender and one receiver (2)
- ❑ Read-modify-write on ETS uses `update_counter` / `update_element` (4)
- ❑ Traversals of busy tables use `ordered_set`, one ETS call, or `safe_fixtable` (4)
- ❑ Anything whose handling depends on a mode is a `gen_statem`, built from a written transition table (5, 13)
- ❑ Values that decide which `gen_statem` events can be handled live in the state, not the data (5)
- ❑ No two servers `call` each other (5)
- ❑ Shared native state is locked or atomic; ideally NIFs are pure (11)

6. What happens when something is slow or unreachable?

- ❑ Every request/reply is tagged with a fresh reference or monitor (2)
- ❑ Asynchronous results carry a reference to the attempt that produced them; stale results are dropped (13)
- ❑ Timeouts are treated as unknown outcomes; retried operations are idempotent (2, 6)
- ❑ Real timers use `send_after/start_timer` or the right `gen_statem` timeout type (5)
- ❑ Per-state timers use `state_timeout`; named timeouts are cancelled or handled in every state (13)
- ❑ Events are postponed only in states the machine will eventually leave (13)
- ❑ Sends to registered names handle the “not registered right now” case (2)
- ❑ No process holds a sibling’s pid across a possible restart (1)
- ❑ `noconnection` is handled as “unknown”, not “dead” (6)
- ❑ No single-leader logic depends only on node monitoring (6)
- ❑ Cross-node updates are acknowledged and idempotent (6)

- ❑ Every global name has a plan for after a partition heals (6)
- ❑ Remote calls have explicit timeouts and don't run inside heavily used callbacks (6)

7. Where does the memory go?

- ❑ Text in hot paths is binaries or iodata, not character lists (3)
- ❑ Output is built as iodata and written once (3)
- ❑ Binary accumulators only append, and only the newest version is used (3)
- ❑ Slices over 64 bytes of large binaries stored long-term are copied, after checking `referenced_byte_size` (3)
- ❑ Long-lived processes that touch large binaries hibernate or garbage-collect regularly (3)
- ❑ Large shared data is in ETS or passed as binaries, not copied per send (2)
- ❑ ETS objects are small enough to copy on every lookup (4)
- ❑ Bulk data between nodes is chunked or sent outside the distribution channel (6)
- ❑ Large data is streamed, not loaded whole (12)

8. Is it fast for the right reason?

- ❑ Frequent ETS queries use the key; other access paths have a secondary index (4)
- ❑ No `tab2list` or unbound-key `match/select` on large tables in hot paths (4)
- ❑ `bag` tables have no hot keys (4)
- ❑ Concurrency options and `persistent_term` are used for measured reasons (4)
- ❑ Performance claims come from profiles or benchmarks of compiled code (10)
- ❑ Benchmarks run for seconds in fresh processes (10)
- ❑ Capacity decisions use scheduler utilisation, not only OS CPU (10)
- ❑ `+bin_opt_info` has been checked on binary-heavy modules (3)

9. Is native code contained?

- ❑ Native code runs as a port unless port overhead is measured and unacceptable (11)
- ❑ Every NIF returns within about 1 ms or runs on the correct dirty scheduler (10, 11)
- ❑ NIFs never keep terms from a process-bound environment (11)
- ❑ Native memory handed to Erlang is always a resource object (11)
- ❑ Port programs log to stderr only; protocols are framed (11)

10. Is it safe from input and from the network?

- ❑ No atoms are created from external input (3, 12)
- ❑ No shell commands are built from input; `spawn_executable` with `args` instead (11)
- ❑ Distribution uses TLS; epmd and distribution ports are firewalled (6)
- ❑ Remote calls use module/function/arguments, not funs (6)

11. Is it configured and deployed correctly?

- ❑ Every runtime dependency is listed in `applications` (7)
- ❑ The system is tested by booting the release, not only in the shell (7)
- ❑ Configuration is read at runtime where it is meant to change; secrets in `config/runtime.exs` (7, 12)
- ❑ `set_env` is not used as a runtime control mechanism (7)
- ❑ Long-running processes are behaviours, or make fully qualified recursive calls (5, 7)

12. Can you see what it's doing?

- ❑ Logging uses `?LOG_*` macros, preferably as structured reports (9)
- ❑ Formatter `depth` / `chars_limit` are set; logger drop counts are monitored (9)
- ❑ Important events are also counted as metrics (9)
- ❑ Production tracing targets specific processes or is rate-limited, and is always stopped (9)
- ❑ Mailbox sizes are read with `message_queue_len` (9)
- ❑ `sys` functions are used only for debugging (9)
- ❑ BEAM-level metrics are exported and `system_monitor` is installed (9)

13. Do the tests prove anything?

- ❑ No sleeping for synchronisation; tests wait on calls, messages or monitors (8)
- ❑ Every test cleans up its processes; no shared registered names (8)
- ❑ Pure functions with wide input spaces have property tests (8)
- ❑ Stateful components have at least one stateful property or model-based test (8)
- ❑ Integration tests boot the real application (8)
- ❑ Dialyzer runs in CI; specs describe real return values (8)
- ❑ Expected values come from requirements, not from the implementation (8)

14. Elixir-specific

- ❑ Processes model state, concurrency or failure; pure logic stays in plain modules (12)
- ❑ `Task.async` only where a task failure should crash the caller; otherwise `Task.Supervisor.async_nolink` (12)
- ❑ No expensive functions passed into `Agent` calls (12)